

CURSO BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GABRIELLY AZEVEDO ZGODA

TRABALHO 1 - SIMULAÇÃO DINÂMICA DE PARTÍCULAS

PINHAIS

2026

GABRIELLY AZEVEDO ZGODA

TRABALHO 1 - SIMULAÇÃO DINÂMICA DE PARTÍCULAS

Trabalho 1 de Projeto de Sistemas Distribuídos, do curso de Bacharelado em Ciência da Computação do Instituto Federal do Paraná, Campus Pinhais, sob a orientação do professor Gabriel Vinicius Canzi Candido.

PINHAIS
2026

Sumário

Introdução	4
Metodologia	4
Ambiente de Teste	5
Regiões paralelizáveis e métricas estimadas	5
Resultados	6
Speedup por número de threads	8
Discussão	9
Conclusão	10
Referências	11

Introdução

A simulação de N-corpos gravitacionais é uma ferramenta central na astrofísica computacional. Ela é usada para estudar a formação e evolução de sistemas estelares e planetários. Esses processos ocorrem ao longo de milhões de anos e só podem ser investigados por simulação, nunca por observação direta. O mesmo princípio matemático (cálculo de forças gravitacionais entre corpos) é aplicado por agências espaciais no planejamento de trajetórias de satélites e sondas, e na previsão de colisões entre asteroides e a Terra.

O desafio computacional deste tipo de simulação é que o cálculo de força entre cada par de partículas cresce de forma quadrática ($O(N^2)$) em relação ao número de corpos: dobrar o número de partículas quadruplica o número de cálculos necessários. Simulações astrofísicas reais frequentemente envolvem milhões de partículas (por exemplo, ao simular uma galáxia inteira), tornando o processamento sequencial inviável em termos de tempo. É por isso que a paralelização é indispensável nesse tipo de aplicação: distribuir os cálculos entre múltiplos núcleos é o que torna essas simulações executadas em um tempo bem menor do que de costume, e é exatamente essa relação entre tamanho do problema e necessidade de paralelismo que este trabalho investiga.

Metodologia

Foram implementadas duas versões do simulador em C++, ambas usando o integrador Velocity Verlet (é mais preciso que o método de Euler para preservar a energia do sistema em órbitas de longo prazo) e a mesma semente de geração aleatória (42), garantindo que ambas simulem exatamente o mesmo cenário físico, o que torna a comparação de tempo justa.

O cálculo de força entre cada par de partículas segue a Lei da Gravitação Universal de Newton: $F = G \times m_1 \times m_2 \div d^2$, onde G é a constante gravitacional, m_1 e m_2 são as massas das partículas, e d é a distância entre elas. Para evitar divisão por zero quando duas partículas se aproximam excessivamente, é somado à distância ao quadrado um pequeno fator de suavização (softening). A partir da força resultante, a aceleração de cada partícula é obtida (aceleração = força $\div$ massa) e aplicada através do integrador Velocity Verlet: a cada passo de tempo, a velocidade é atualizada em duas metades (antes e depois de mover a posição), com a aceleração sendo recalculada no meio do processo, evita o acúmulo de erro de energia que ocorreria com o método de Euler simples, importante para manter órbitas estáveis ao longo dos 20.000 passos simulados. As partículas iniciais são geradas com uma posição aleatória e uma velocidade calculada para resultar em órbita circular estável ao redor de um corpo central fixo ($v = \sqrt{(G \times \text{massa_central} \div \text{raio})}$), usando uma semente fixa para garantir reprodutibilidade entre as execuções.

Ambas as versões (simulacao_gravitacional.cpp e simulacao_paralela.cpp) calculam a aceleração de cada partícula considerando a influência gravitacional de todas as demais, ou seja, nenhuma das duas aproveita a simetria da 3ª Lei de Newton (ação e reação) para economizar cálculos. Essa escolha foi feita para isolar o efeito real da paralelização: ao garantir

que as duas versões realizam exatamente o mesmo volume de operações, qualquer diferença de tempo medida pode ser atribuída à distribuição do trabalho entre threads, e não a uma diferença na quantidade de cálculos realizados.

Versão sequencial (simulacao_gravitacional.cpp): percorre todas as partículas em um único processo, calculando a aceleração de cada uma em relação às demais.

Versão paralela (simulacao_paralela.cpp, com OpenMP): distribui esse mesmo cálculo entre múltiplas threads, uma partícula por thread a cada instante, o que também evita condições de corrida (duas threads escrevendo na mesma partícula ao mesmo tempo), já que cada thread só escreve nos dados da partícula que lhe foi atribuída.

A explicação linha a linha de como cada arquivo de código funciona (incluindo o formato do CSV gerado e o funcionamento do visualizador HTML) está documentada no arquivo README.md que acompanha o código-fonte.

Ambiente de Teste

Todas as medições foram feitas na mesma máquina pessoal da estudante, na qual tem essas especificações:

Tabela de especificações do hardware

Especificação	Característica
Processador	Intel Core i7-1355U (13ª geração)
Threads lógicas disponíveis	12
Memória RAM	7,6 GB
Sistema operacional	Windows 11 com WSL2 (Ubuntu)
Compilador	g++ com flag -O2 (otimização habilitada) e -fopenmp na versão paralela

Fonte: Elaborado pela autora (2026).

O i7-1355U possui arquitetura híbrida (núcleos de performance e núcleos de eficiência), o que significa que as 12 threads lógicas não têm desempenho idêntico entre si.

Regiões paralelizáveis e métricas estimadas

A região paralelizável do algoritmo é o cálculo de aceleração (calcularAceleracoesParalelo): para cada partícula, é necessário considerar a influência gravitacional de todas as demais, o que dá a essa etapa complexidade $O(N^2)$, ela cresce muito mais rápido que o restante do programa conforme N aumenta. As etapas de atualização de

velocidade e posição ($O(N)$, lineares) e a escrita no arquivo CSV permanecem sequenciais, pois têm custo desprezível perto do cálculo de força e não representam o gargalo do programa.

Para estimar o speedup teórico, foi usada a Lei de Amdahl, que relaciona a fração do programa que pode ser paralelizada (P) com o número de threads (n):

$$S(n) = 1 / ((1-P) + P/n)$$

Como o cálculo de força domina o tempo total de execução para os tamanhos de N testados (o componente $O(N^2)$ cresce muito mais rápido que o $O(N)$ das demais etapas), estimou-se $P = 0,95$, ou seja, assumiu-se que 95% do tempo de execução está na região paralelizável, restando apenas 5% de trabalho estritamente sequencial. Essa é uma estimativa conservadora baseada na natureza assintótica do algoritmo, não uma medição direta por profiling.

Tabela - Estimativa do Speedup e aplicação real

Threads (n)	Speedup teórico (Amdahl, $P=0.95$)	Speedup real observado ($N=5000$)
1	1.00x	1.48x
2	1.90x	2.47x
4	3.48x	2.34x
12	7.74x	3.11x

Fonte: Elaborado pela autora (2026).

O speedup real com 12 threads ainda ficou abaixo do estimado pela Lei de Amdahl (3.11x vs. 7.74x), reforçando que fatores como o overhead de sincronização a cada passo de tempo e a arquitetura híbrida do processador têm impacto relevante. Curiosamente, com 2 threads o speedup real superou a estimativa teórica (2.47x vs. 1.90x), um resultado que é possível quando a divisão dos dados entre threads também melhora o aproveitamento da memória cache do processador, efeito que o modelo simples de Amdahl não contempla.

É interessante que mesmo com 1 thread (onde o speedup teórico é necessariamente 1.00x, por não haver paralelismo), o speedup real observado foi de 1.48x. Esse resultado também é discutido na seção de Resultados como possível efeito de condições de teste não perfeitamente controladas entre as duas versões.

Resultados

Cada combinação de número de partículas (N) e número de threads foi executada uma vez, medindo o tempo com `std::chrono`. Speedup = tempo sequencial ÷ tempo paralelo; Eficiência = Speedup ÷ número de threads.

Tabela - Número de partículas e número de threads

N	Threads	Tempo Sequencial (s)	Tempo Paralelo (s)	Speedup	Eficiência
500	1	51.16	22.27	2.30x	230%
500	2	51.16	15.18	3.37x	169%
500	4	51.16	24.44	2.09x	52%
500	12	51.16	21.08	2.43x	20%
1000	1	150.09	111.35	1.35x	135%
1000	2	150.09	79.46	1.89x	95%
1000	4	150.09	94.26	1.59x	40%
1000	12	150.09	50.75	2.96x	25%
2000	1	495.38	338.04	1.47x	147%
2000	2	495.38	201.68	2.46x	123%
2000	4	495.38	329.73	1.50x	38%
2000	12	495.38	172.44	2.87x	24%
5000	1	2902.79	1967.89	1.48x	148%
5000	2	2902.79	1177.18	2.47x	123%
5000	4	2902.79	1240.97	2.34x	58%
5000	12	2902.79	932.51	3.11x	26%

Fonte: Autora (2026).

Em três dos quatro tamanhos testados (N=1000, 2000 e 5000), 12 threads produziu o menor tempo de execução. A exceção é N=500, onde 2 threads foi a configuração mais rápida (15.18s). Isso aconteceu provavelmente porque com um volume de cálculo pequeno, o custo de coordenar 12 threads não compensa frente ao ganho, e a variação de desempenho entre os núcleos do processador (discutida adiante) tem maior peso relativo.

Embora as duas versões agora realizem exatamente a mesma quantidade de cálculos, a versão paralela com apenas 1 thread apresentou tempos consistentemente menores que a sequencial (de 26% a 56% mais rápida, dependendo de N). Esse foi um resultado que não deveria ocorrer, já que sem paralelismo real, as duas deveriam ter desempenho equivalente. Ambas as baterias de teste foram executadas com a máquina sem uso simultâneo para outras tarefas, o que descarta interferência direta de multitarefa como causa. Entre as explicações mais prováveis estão: (1) a execução sequencial completa levou cerca de 60 minutos contínuos de processamento intenso, o que pode ter causado redução de desempenho por aquecimento do processador (throttling térmico), comum em notebooks finos sob carga prolongada; (2) os

dois conjuntos de testes foram realizados em momentos diferentes (madrugada vs. período diurno), sujeitos a variações não controladas do sistema operacional e do ambiente WSL; e (3) cada configuração foi medida uma única vez, sem repetições que permitissem calcular uma média e descartar outliers. Não foi possível isolar com certeza qual desses fatores foi dominante, o ideal seria realizar múltiplas repetições intercaladas entre as duas versões, sob as mesmas condições térmicas e de carga do sistema, para isolar com precisão o efeito real da paralelização.

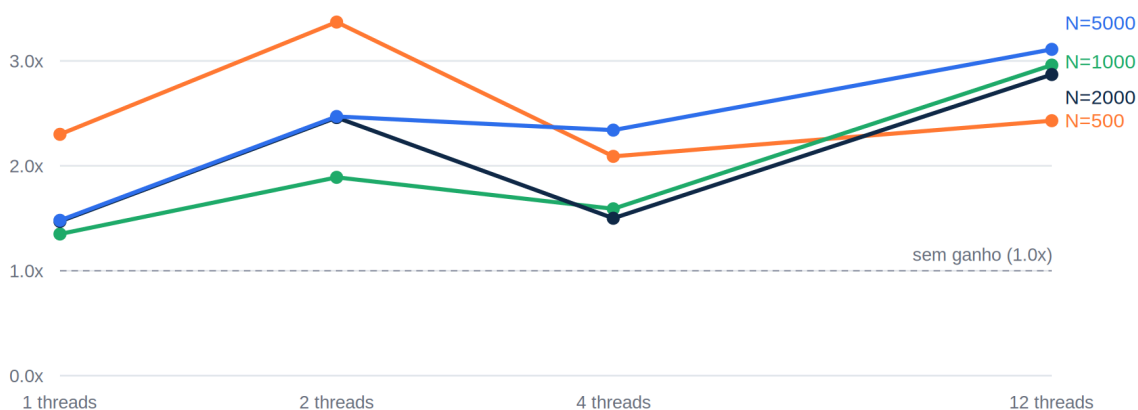
Além disso, foi perceptível que 2 threads foram mais rápidas do que 4 em diversos casos, comportamento esse que aparece de forma consistente em $N=500$, $N=1000$, $N=2000$ e $N=5000$, então não parece ser apenas ruído de medição. A explicação mais provável está na arquitetura híbrida do processador utilizado (Intel Core i7-1355U): ele possui 2 núcleos de performance (P-cores, com hyperthreading, 4 threads lógicas) e 8 núcleos de eficiência (E-cores, mais lentos, 1 thread lógica cada), totalizando 12 threads. Com apenas 2 threads solicitadas, é provável que o sistema operacional as aloque nos 2 núcleos de performance, os mais rápidos disponíveis. Já com 4 threads, pelo menos algumas delas provavelmente são atribuídas aos núcleos de eficiência (mais lentos) ou dividem um mesmo núcleo de performance via hyperthreading que, para código com muitas operações de ponto flutuante como este, tende a gerar disputa pelos mesmos recursos de execução, em vez de dobrar o desempenho. Como a região paralela (`#pragma omp parallel for`) possui uma barreira de sincronização implícita ao final, ou seja, todas as threads precisam terminar antes do próximo passo de tempo começar, o tempo total da execução é limitado pela thread mais lenta, não pela mais rápida. Isso significa que basta uma thread cair num núcleo de eficiência para que todo o grupo seja "segurado" por ela, degradando o desempenho mesmo com mais threads em uso. Uma forma de investigar essa hipótese em trabalhos futuros seria fixar a afinidade das threads aos núcleos de performance (com `OMP_PROC_BIND` ou `GOMP_CPU_AFFINITY`) e comparar o resultado.

Speedup por número de threads

A tabela de resultados apresentada anteriormente contém 16 combinações de N e número de threads, o que dificulta enxergar tendências apenas olhando os números. O gráfico a seguir isola a métrica de speedup (razão entre o tempo sequencial e o tempo paralelo) e a organiza por número de threads, com uma linha para cada tamanho de N testado. Permitindo visualizar como o ganho da paralelização se comporta à medida que threads e partículas aumentam.

Speedup cresce com mais threads, exceto em N=500

Speedup máximo de 3.37x, com 2 threads e N=500



Fonte: Elaborado pela autora (2026).

O gráfico acima evidencia um padrão parcialmente diferente do esperado. Em três dos quatro tamanhos testados (N=1000, 2000 e 5000), o speedup máximo ocorre com 12 threads, crescendo de forma consistente com o tamanho do problema de 2.96x em N=1000 até 3.11x em N=5000, confirmando que o ganho da paralelização se torna mais relevante conforme o volume de cálculo aumenta. A exceção é N=500, onde o speedup máximo (3.37x) ocorre com apenas 2 threads, superando inclusive o resultado obtido com 12 threads (2.43x). O que sugere que para volumes de cálculo pequenos, o overhead de coordenar um número maior de threads pode superar o ganho adicional de paralelismo, tornando uma quantidade menor de threads mais vantajosa.

Assim como discutido anteriormente, nota-se também uma queda no speedup com 4 threads em relação a 2 threads, presente em todos os quatro tamanhos de N. Esse comportamento é discutido em detalhe na seção de Discussão, e está relacionado à arquitetura híbrida do processador utilizado.

Discussão

Embora 12 threads produza o menor tempo na maioria dos casos, a eficiência despenca de ~135-230% (com 1 thread) para 20-26% (com 12 threads). Isso é esperado: dobrar o número de threads nunca dobra a velocidade na prática, por causa do overhead de criar e sincronizar threads e da divisão de memória entre núcleos.

Porém, a versão paralela rodando com apenas 1 thread foi consistentemente mais rápida que a sequencial em todos os quatro tamanhos de N testados, um resultado que não deveria acontecer, já que com 1 thread as duas versões realizam exatamente o mesmo cálculo e deveriam ter tempos equivalentes. Ambas as baterias de teste foram executadas com a máquina sem uso simultâneo para outras tarefas, o que descarta interferência direta de multitarefa como causa.

Entre as explicações mais prováveis estão: (1) a execução sequencial completa levou cerca de 60 minutos contínuos de processamento intenso, o que pode ter causado redução de desempenho por aquecimento do processador (throttling térmico), comum em notebooks finos sob carga prolongada; (2) os dois conjuntos de testes foram realizados em momentos diferentes (madrugada vs. período diurno), sujeitos a variações não controladas do sistema operacional e do ambiente WSL; e (3) cada configuração foi medida uma única vez, sem repetições que permitissem calcular uma média e descartar outliers.

Não foi possível isolar com certeza qual desses fatores foi dominante, o ideal seria realizar múltiplas repetições intercaladas entre as duas versões, sob as mesmas condições térmicas e de carga do sistema, para isolar com precisão o efeito real da paralelização.

O speedup obtido cresce conforme N aumenta, chegando a 3.11x em $N=5000$ com 12 threads, o que confirma que o problema escala bem: quanto maior o volume de cálculo, mais o benefício da paralelização supera o overhead de gerenciá-la. Isso sugere que, para os tamanhos de N típicos de simulações astrofísicas reais (milhares a milhões de partículas), o ganho da paralelização tende a ser ainda mais expressivo.

Ainda assim, o speedup real com 12 threads permanece abaixo do estimado pela Lei de Amdahl (3.11x contra 7.74x teórico), o que indica que o overhead de sincronização a cada passo de tempo e a arquitetura híbrida do processador (discutida anteriormente) têm impacto relevante, mesmo quando as duas versões realizam exatamente a mesma quantidade de cálculos.

Considerando isso, a solução proposta demonstra corretamente os princípios de paralelização com OpenMP e evidencia uma relação clara entre tamanho do problema e benefício da paralelização, mas também revela que fatores de hardware e de sincronização impõem um teto ao ganho possível, mesmo numa implementação paralela bem construída.

Conclusão

A paralelização com OpenMP se mostrou consistentemente vantajosa ao utilizar múltiplas threads, atingindo speedup de até $3.11\times$ para 5.000 partículas, e a vantagem cresce com o tamanho do problema, o que confirma a relação entre a complexidade $O(N^2)$ do algoritmo e a necessidade de paralelismo discutida na Contextualização. Em contrapartida, a eficiência decresce com mais threads, refletindo o overhead de sincronização inerente à paralelização e, possivelmente, a arquitetura híbrida do processador utilizado nos testes.

Este trabalho também evidenciou a importância de um ambiente de teste controlado para medições de desempenho confiáveis: a diferença nas condições de execução entre as versões sequencial e paralela introduziu ruído que distorceu parte dos resultados, especialmente nas comparações envolvendo 1 thread. Como trabalho futuro, seria valioso repetir todas as medições sob condições idênticas de carga do sistema (preferencialmente com a máquina ociosa), testar tamanhos de N ainda maiores, e investigar o efeito da afinidade de threads aos núcleos de performance do processador (via `OMP_PROC_BIND` ou `GOMP_CPU_AFFINITY`) sobre a estabilidade dos resultados.

Referências

- ARAÚJO, Mariana de. **Lei da gravitação universal**. Revista de Ciência Elementar, v. 1, n. 01, p. 013, 2013. DOI: <https://doi.org/10.24927/rce2013.013>. Disponível em: https://web.archive.org/web/20220301044515id_/https://rce.casadasciencias.org/rceapp/static/docs/artigos/2013-013.pdf. Acesso em: 13 set. 2026.
- HENTZ, Agenor; BRUNET, Leonardo. **Notas de aula: Métodos Computacionais da Física**. Instituto de Física, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre. Disponível em: https://www.if.ufrgs.br/~leon/metcomp/notas_de_aula.pdf. Acesso em: 13 set. 2026.
- NASCIMENTO, Jefferson Oliveira do; NEIDE, Italo Gabriel; GONZATTI, Sônia Eliza Marchi; MORET, Marcelo Albano. **A modelagem e a simulação computacional como ferramentas tecnológicas no ensino de física**. Revista Signos, Lajeado, ano 38, n. 1, 2017. ISSN 1983-0378. DOI: <http://dx.doi.org/10.22410/issn.1983-0378.v38i1a2017.1378>.